Report: Handwritten Sudoku Reader and Solver

Team:

Yussuf Akram	*320240111*
Youssef Hamdy	*320240090*

1. Summary

We built a system that reads a Sudoku drawn as a 9×9 mosaic of handwritten MNIST digits, repairs the digits a neural network misreads, and solves the puzzle. With up to 3 misread givens it recovers the exact original solution in 87–95 % of 100 puzzles per level; with 4 misreads, which exceeds the 3-correction budget by design, it recovers none.

A convolutional network (CNN) reads each cell and gives a probability for every digit 1–9. If the reading breaks the puzzle, AC-3 detects the contradiction. A search over sets of corrections (UCS, Greedy or A*) then tries the cheapest changes first, where a change costs how much less likely the new digit is. A backtracking CSP solver tests each corrected puzzle, and the first one with exactly one solution wins. If no set of at most 3 corrections works, the system answers "No solution: puzzle unreadable" and lists the 5 least confident cells.

Main results (seed 7, MNIST test images placed in generated puzzles):

- **Network:** the CNN reads 98.49 % of the 9,020 test digits correctly, against 92.24 % for logistic regression (Section 4).
- **Recovery:** NN-guided correction restores the exact solution for 95 %, 89 % and 87 % of puzzles with 1, 2 and 3 misreads. With no correction the rate is 0 %, and random correction reaches 28 % at 1 misread and 0 % beyond (Section 7).
- **Search:** A* always returns the same cost as UCS and expands 1.8–4.7 times fewer nodes at 1–3 misreads, but UCS is faster in seconds on this small space (Section 6).
- **CSP:** on 25-given puzzles forward checking cuts backtracking assignments from about 44,600 to 13,600, AC-3 to 9,460, and maintaining arc consistency (MAC) to about 270 (Section 5).
- **Limit:** a consistency check cannot tell a wrong but uniquely solvable puzzle from the right one, so some answers are confidently wrong (Section 8).

2. Problem and data

The task. The input is a 252×252 grey-scale image: a 9×9 grid of 28×28 tiles, each tile an MNIST digit or blank. The output is the solved 9×9 grid, or the message "No solution: puzzle unreadable" with the five least confident cells. The network sometimes misreads a digit, and one misread given usually breaks the puzzle: two equal digits appear in a row, or the puzzle gets no solution or several. The system may

change at most 3 cells. In the brief's example the network reads a 7 as a 1 (52 % against 45 %), two 1s appear in one row, and changing that cell to 7 is cheap because the network was nearly as sure about 7.

Real and generated data. The digit images and labels are real: MNIST has 60,000 training and 10,000 test images from different writers (LeCun et al., 1998). Everything else is generated by our code from one seed (`SEED = 7` in `sudoku/config.py`). This is not photographed Sudoku.

Experiments 1 and 3 and the recovery study use 30-given puzzles; Experiment 2 uses 25, 30 and 35.

Why 45,075 and not 50,000 training images. The brief asks for a 50,000 / 10,000 split, but only 54,077 of the 60,000 training images are digits 1–9, so both parts cannot be filled after dropping 0. We split all 60,000 images first and drop the zeros inside each part.

Hard-digit injection. The CNN reads 98.49 % of the test digits correctly, so random mosaics would contain almost nothing to correct. We therefore inject hard digits by a documented rule. The *hard pool* holds the test images that the reader gets wrong or reads with top-1 probability below 0.6, and whose true digit is among its top three, so a correction can reach it: 176 images. Of these, 133 are outright misreads, between 8 and 32 per digit. To build a puzzle at misread level k, we replace the image in k randomly chosen given cells by a misread image of the same digit, and give every other given an image the reader gets right. Level k therefore contains exactly k misread cells (levels 0–4). The brief's literal rule, which injects from the whole hard pool and leaves other cells unrestricted, is kept behind a configuration flag. We use the exact version because a recovery rate per misread level only means something if level k really has k errors.

3. System design

Three components each have one job and meet only in the pipeline function (`sudoku/integration/pipeline.py`): the network turns pixels into probabilities, the search turns probabilities into candidate corrections, and the CSP decides whether a corrected puzzle is valid.

1. **Read.** A brightness rule marks the empty tiles. The CNN gives every other tile a probability for each digit 1–9, after temperature scaling (Section 4). The read grid holds the most probable digit of each tile.
2. **Check.** AC-3 and a duplicate-givens check test the read grid. If it is consistent and has exactly one solution, we solve it and report status "solved".
3. **Correct.** Otherwise a search over correction sets runs, A* by default. The CSP tests each candidate set, and the first goal state the search pops wins: status "corrected", with a correction log of cell, read digit, new digit and cost.
4. **Give up cleanly.** If no set of at most 3 corrections works, the status is "unreadable": the message "No solution: puzzle unreadable" and the five least confident cells with their top-3 probabilities.

The "NN off" mode skips step 3 and solves the read grid as it is. It is the comparison for the whole system. Figure 1 shows the three stages for one puzzle.

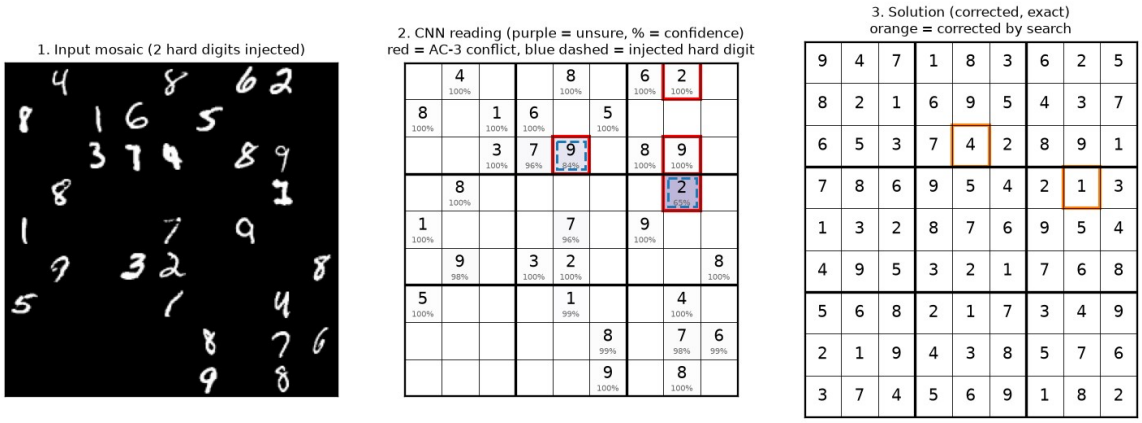


Figure 1. Demo puzzle (seed 7, 2 misreads). Left: the input mosaic with two injected hard digits. Middle: the CNN reading with confidence; red boxes mark the duplicates AC-3 finds, dashed blue boxes the injected hard digits. Right: the solution after two corrections (orange), identical to the original.

Design choices

- **Cost of a correction.** Changing a cell from its read digit r to digit d costs log p(r) − log p(d). It is never negative because r is the most probable digit. Summed over cells, the cost is minus the log of how much less likely the corrected reading is than the original, if cells are independent, so cheapest-first search tries the most likely explanation first.
- **Which corrections are allowed.** Only the 15 least confident given cells may change, each to one of its 3 most probable other digits, with at most one change per cell and at most 3 changes. That gives 1 + 45 + 945 + 12,285 = 13,276 states in total. Section 8 reports that no misread cell fell outside the 15.
- **Goal test.** A corrected puzzle must have exactly one solution (the brief's extension), not just at least one, because a puzzle with several solutions is not a valid Sudoku. The brief's minimum version, at least one solution, is available through a flag, and the live demo can show both.
- **Fair CSP comparison.** In Experiment 2 all CSP modes use the same fixed row-major variable order. With a minimum-remaining-values (MRV) order computed from the assignment, backtracking, +FC and +FC+AC-3 make identical assignments, so the comparison would show nothing. The goal test inside the search uses MRV with FC, AC-3 and MAC instead, because it can run thousands of times per puzzle.
- **Calibrated probabilities.** Temperature scaling, fitted on validation data, makes the probabilities behind the costs calibrated (Section 4).

4. The neural network

The CNN reads 98.49 % of the 9,020 test digits correctly, against 92.24 % for logistic regression. Both are well calibrated, with a calibration error below 0.014, and temperature scaling improves the CNN's calibration while worsening the baseline's.

Models. The CNN has two convolution blocks (3×3 kernels, 16 and 32 filters, ReLU, 2×2 max-pooling), one fully connected layer of 128 units and a 9-way output. It trains with Adam (learning rate 0.001, batch size 128, cross-entropy) for 3 epochs on the 45,075 training images, about 20 seconds on a CPU.

The baseline is multinomial logistic regression on the 784 raw pixels (scikit-learn), trained on the same 45,075 images in about 39 seconds, so the comparison is fair.

Model (MNIST test, 9,020 digits 1–9)	Accuracy	NLL	ECE (15 bins)	Temperature T
CNN	98.49 %	0.0425	0.0055	1
CNN + temperature scaling	98.49 %	0.0406	0.0039	0.868
Logistic regression	92.24 %	0.2818	0.0051	1
Logistic regression + temperature scaling	92.24 %	0.2798	0.0133	1.076

Calibration. The search turns probabilities into costs, so they must mean what they say. We measure the expected calibration error (ECE, 15 equal-width confidence bins) and fit one temperature T on the validation set (Guo et al., 2017) by minimising the negative log-likelihood (NLL) of softmax(z / T), where z are the network's raw scores. T does not change which digit is most probable, so accuracy is unchanged. For the CNN, T = 0.868 sharpens the probabilities: the network was slightly under-confident, and ECE falls from 0.0055 to 0.0039. For logistic regression T = 1.076 lowers NLL a little (0.2818 to 0.2798) but raises ECE from 0.0051 to 0.0133. The two measures answer different questions, and with 9,020 test images an ECE of this size is small and noisy, so we do not read much into the baseline's rise. The reliability diagrams in Figure 2 follow the diagonal for both models, with noisy bars at low confidence where few images fall.

Confusion matrices. The CNN's 136 errors are spread thinly: the most common are a 7 read as 9 (16 images), a 4 read as 9 (11) and a 7 read as 2 (9). Logistic regression makes 700 errors, led by a 5 read as 3 (36) and a 7 read as 9 (34). Both matrices are saved in results/nn/ (confusion_cnn.png and confusion_logreg.png).

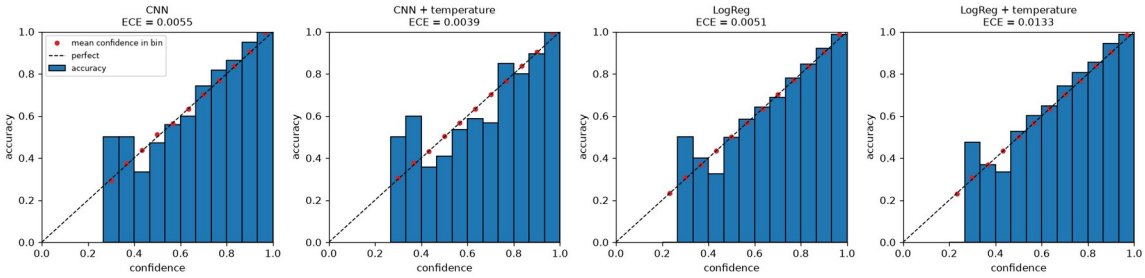


Figure 2. Reliability diagrams on the MNIST test digits: accuracy in each of 15 confidence bins against the diagonal of perfect calibration, for the CNN and logistic regression, before and after temperature scaling.

5. The CSP solver

Forward checking and AC-3 (Mackworth, 1977) each cut the search for a Sudoku solution, and maintaining arc consistency during search (MAC) cuts it further. On 25-given puzzles the mean number of assignments falls from about 44,600 for plain backtracking to 13,600 with forward checking, 9,460 with AC-3 added, and 270 with MAC.

Formulation. There are 81 variables, one per cell, with domain 1–9; a given has a domain of one value. In every row, column and 3×3 box the cells must differ. As binary constraints this gives 1,620 directed arcs, because each cell has 20 peers.

Switchable parts. The solver is called as `solve(grid, use_fc, use_ac3, use_mac)`, and backtracking is always on.

- **Backtracking (BT):** assign cells one at a time in a fixed order and undo an assignment when it fails.
- **Forward checking (FC):** after an assignment, remove that digit from the domains of the cell's unassigned peers; an empty domain fails immediately.
- **AC-3:** run once before the search, revising arcs until no domain changes. An empty domain means a contradiction, for instance two equal givens in one row, found without any search. Otherwise the domains are smaller when the search starts.
- **MAC (extension):** run AC-3 again after every assignment.

Experiment 2. We solved 100 puzzles per given count (5 seeds of 20 puzzles) with the same static row-major order in every mode. The plain backtracking mean at 25 givens is a lower bound, because 10 of its 100 puzzles stopped at the 200,000-assignment limit.

Mean assignments per puzzle	BT	BT + FC	BT + FC + AC-3	+ MAC (extension)	Timeouts per 100 (BT / FC / FC+AC-3 / MAC)
25 givens	44,581	13,633	9,463	271	10 / 1 / 1 / 0
30 givens	5,523	1,266	540	60	0 / 0 / 0 / 0
35 givens	587	205	96	47	0 / 0 / 0 / 0

Each switch lowers the number of assignments at every puzzle size. Compared with plain BT, FC removes 69 %, 77 % and 65 % of the assignments at 25, 30 and 35 givens; adding AC-3 removes a further 31 %, 57 % and 53 % compared with FC alone. Backtracks follow assignments closely (44,528 backtracks for 44,581 assignments for BT at 25 givens), because almost every assignment fails.

Fewer assignments do not always mean less time. AC-3's preprocessing costs about 2,400 to 2,900 revisions per puzzle. At 25 and 30 givens BT + FC + AC-3 is the fastest of the three brief modes (0.032 s against 0.045 s for BT + FC at 25 givens, 2.8 ms against 4.2 ms at 30). At 35 givens it is slower than BT + FC (1.5 ms against 0.8 ms), because those puzzles are easy and the AC-3 cost is not repaid. Timed-out runs count the work done up to the cap, so the BT averages at 25 givens are lower bounds. Every solved run returned the generator's solution.

6. Correction search

A* (Hart et al., 1968) finds the same cheapest correction set as UCS while expanding 1.8 to 4.7 times fewer nodes at 1–3 misreads. UCS is nevertheless faster in seconds on a space this small, and Greedy is quick but sometimes returns a dearer set.

Search problem. A state is a set of corrections {(cell, new digit)}, at most one per cell and at most 3 in all. The start state is the empty set, which trusts the network. An action corrects one more of the 15

least confident given cells to one of its three most probable other digits. Changing cell i from its read digit r to digit d costs

$$c(i, r \to d) = \log p_i(r) - \log p_i(d) \ge 0$$

and a path costs the sum of its steps. The goal test (Section 3) asks that the corrected puzzle has no duplicate givens, passes AC-3 and has exactly one solution.

Algorithms. One best-first graph search (Russell and Norvig, 2020) runs with three priorities: UCS orders states by g, the path cost so far; Greedy by h alone; A* by f = g + h. The goal test runs when a state is popped, not when it is generated, so UCS and A* return the cheapest goal. A set of corrections reached in two different orders is expanded once.

Heuristic. A conflict is a set of givens in one row, column or box that show the same digit. No goal state has a conflict, so at least one cell of each conflict must still change. For a conflict C, let m(C) be the cheapest alternative cost among its cells that can still change. Then

$$h(S) = \max_{C \in \mathrm{conflicts}(S)} m(C), \qquad h(S) = 0 \text{ if there is no conflict}$$

and h is infinite when a conflict has no cell left to change; such a child is pruned. The heuristic never overestimates, because every goal below S must fix every conflict, and fixing C costs at least m(C). It takes the maximum and not the sum because two conflicts can share a cell, and a sum would count that one change twice. The heuristic is also consistent, so A* with a closed set is optimal. Tests confirm both properties by brute force over every state of a small problem, and confirm that UCS and A* return the brute-force optimum.

Worked example. In the demo (seed 7, 2 misreads, Figure 1) the CNN reads the two injected hard digits as a 9 (84 %) and a 2 (65 %); the true digits are 4 and 1. The 9 duplicates a 9 in its row and the 2 duplicates a 2 in its column. A* pops four states (Figure 3, left): the empty set with h about 1.75; a single correction of cost about 0.62; a second single correction of cost about 1.75; and the goal, the pair of corrections with total cost about 2.37, which restores the exact solution. On this puzzle UCS and A* expand 3 states and generate 129, and Greedy expands 2 and generates 87.

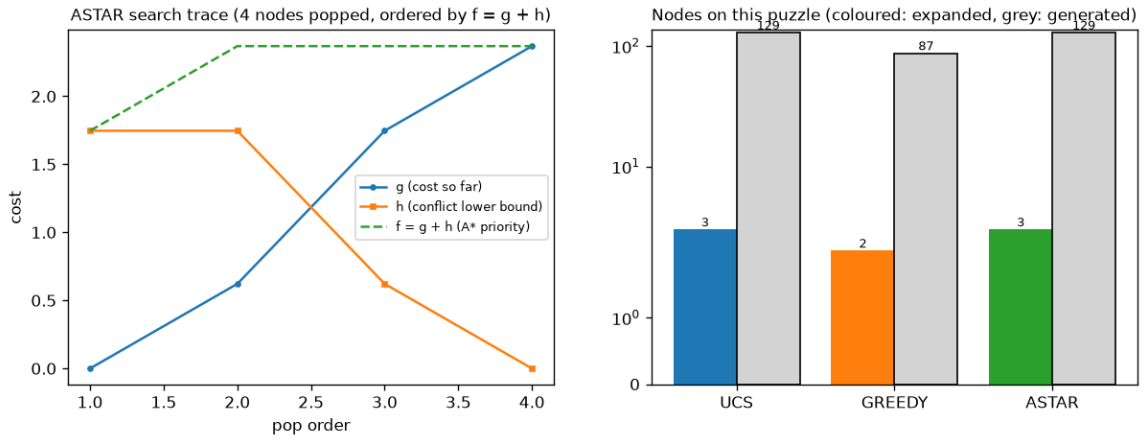


Figure 3. Demo puzzle. Left: g, h and f = g + h of the four states A pops. Right: nodes expanded (colour) and generated (grey) by UCS, Greedy and A* on this puzzle.*

Experiment 1. We ran UCS, Greedy and A* on 100 puzzles per misread level (5 seeds of 20 puzzles, 30 givens). With 0 misreads no search is needed: no algorithm expands a node, and one goal test confirms the reading.

Misreads	UCS: nodes expanded (generated)	Greedy	A*	Exact recovery UCS / Greedy / A*
1	2.0 (87)	1.2 (52)	1.1 (51)	95 / 93 / 95 %
2	8.8 (342)	6.4 (160)	3.5 (147)	89 / 84 / 89 %
3	52.5 (1,530)	23.3 (627)	11.2 (424)	87 / 77 / 87 %
4	8,195 (25,937)	2,126 (10,113)	2,152 (10,687)	0 / 0 / 0 %

A* returned the same total cost as UCS on every puzzle and expanded 1.8, 2.5, 4.7 and 3.8 times fewer nodes at 1, 2, 3 and 4 misreads. Greedy has no optimality guarantee: on the 440 puzzles that it and UCS both solved, it returned a dearer correction set 26 times (mean cost at 3 misreads: 5.32 against 4.55 for A*), and it recovers fewer puzzles at 2 and 3 misreads (84 % and 77 %).

Part of the saving is pruning, not guidance. Children with h infinite are never queued: for A* at 3 misreads, 174 of the 424 generated children (41 %) are pruned, while UCS prunes none. And fewer nodes do not mean less time. A* computes h for every child, which scans all 27 units, so it is slower per node: at 3 misreads UCS takes 0.015 s and A* 0.028 s, and at 4 misreads UCS takes 0.17 s and A* 0.36 s although A* expands 3.8 times fewer nodes. With at most 13,276 states, node counts favour A* and wall-clock time favours UCS. At 4 misreads UCS must test all 13,276 states on the 60 puzzles it declares unreadable; the other 40 end earlier, with a wrong solution (Section 8).

7. Integration and results

The network's probabilities decide the outcome. With NN-guided correction the system recovers the exact solution in 100 %, 95 %, 89 % and 87 % of puzzles with 0, 1, 2 and 3 misreads, while no correction reaches 0 % and random correction 28 % at best.

Pipeline and outputs. `pipeline(mosaic)` returns a status ("solved", "corrected" or "unreadable"), the read grid with its probabilities, the correction log and the solution. The correction log of every evaluated puzzle is saved in `results/experiments/correction_log.csv`: 685 corrections on 336 puzzles (A*). Of these, 94.9 % set the true digit and 97.5 % are on a cell where a hard digit was injected. The live demo `run_demo.py` prints the search trace, the CSP counters and the NN on/off comparison.

Exact recovery (the P7 deliverable). "Exact recovery" means the returned grid equals the generator's original solution. The study uses A* with the exactly-one-solution goal and 100 puzzles per misread level (5 seeds of 20 puzzles). Counts are puzzles out of 100.

Misreads	Exact recovery (mean ± std over 5 seeds)	Exact, no correction needed	Exact after correction	Wrong unique solution	Unreadable
0	100 %	100	0	0	0

Misreads	Exact recovery (mean ± std over 5 seeds)	Exact, no correction needed	Exact after correction	Wrong unique solution	Unreadable
1	95 % ± 5	0	95	5	0
2	89 % ± 7	0	89	11	0
3	87 % ± 8	0	87	13	0
4	0 %	0	0	40	60

At 1–3 misreads no puzzle ends "unreadable": the search always finds a corrected puzzle with exactly one solution, but sometimes the wrong one (Section 8). At 4 misreads exact recovery is impossible within the 3-correction budget, so 60 puzzles end "unreadable" and 40 end with a wrong solution.

Experiment 3: NN on, NN off and random. The table gives exact recovery in % over 100 puzzles per level.

Misreads	NN-guided (A*)	No correction (NN off)	Random correction	Random within candidates (extension)	NN-guided, no temperature (extension)
0	100	100	100	100	100
1	95	0	28	66	95
2	89	0	0	9	89
3	87	0	0	0	87
4	0	0	0	0	0

With the network's help switched off, a single misread already means the original solution is never returned (Figure 4). The two random arms get up to 201 goal tests per puzzle (the empty set plus 200 random draws), while A* needs on average 2.1, 4.5 and 12.2 (at most 48) at 1, 2 and 3 misreads. Random correction may change any given cell; "random within candidates" may only use the same 15 cells and top-3 alternatives as the search. The gap between that arm and NN-guided correction (66 against 95 %, 9 against 89 %, 0 against 87 %) is the value of trying corrections in order of network cost, beyond what the candidate cut-off already gives.

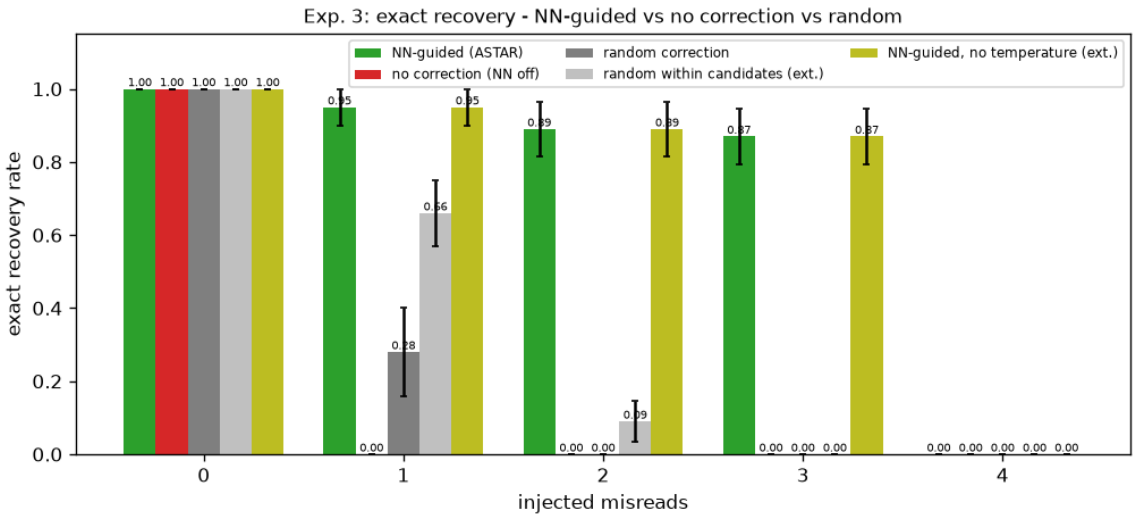


Figure 4. Experiment 3: exact recovery for NN-guided correction, no correction, random correction, random within candidates and NN-guided without temperature scaling. Error bars are the standard deviation over 5 seeds.

Temperature scaling does not change recovery. The step cost of a correction, log p(r) − log p(d), equals (z_r − z_d) / T for raw scores z, so every cost is multiplied by the same factor 1/T and the order of correction sets cannot change. The arm without temperature scaling recovers exactly the same percentages at every level. Calibration matters for the probabilities we report (Section 4), not for which correction wins.

8. Limits

The results show that cost-ordered search works on clean synthetic puzzles. They do not show that the system is safe to trust, or that it would work on photographed Sudoku.

- **Wrong answers that look right.** A uniqueness check cannot tell a wrong but uniquely solvable puzzle from the right one. At 1 misread, 4 of the 5 failures are puzzles where the misread digit gave another valid, uniquely solvable puzzle, so no correction was even tried. At 2 and 3 misreads (11 and 13 failures) the search found a different correction set whose puzzle is also uniquely solvable. At 4 misreads, 40 puzzles end with such a wrong solution. The system reports all of these as solved or corrected, with no warning. In every puzzle all misread cells were among the 15 candidates, so the candidate cut-off did not cause these failures.
- **4 misreads are out of reach by design.** The brief allows at most 3 corrections, so 0 % at level 4 is a property of the budget and says nothing about the algorithms.
- **Clean, synthetic data.** The digits are real, but they sit in a perfectly aligned grid on a black background. There is no photograph, no grid detection, and no skew or noise. The injected hard digits are hard for this particular CNN, which trains for only 3 epochs.
- **A small hard-digit pool.** The misread pool has 133 images (8 to 32 per digit), so the same hard images recur across the 100 puzzles of a level. A different network would give a different pool and different recovery rates.

- **What ± covers.** The five "seeds" are five slices of 20 puzzles from one puzzle bank made with one generator seed, and one trained CNN. The standard deviations show variation over puzzles and image draws, not over retrained networks. With 100 puzzles, a recovery rate near 90 % has a 95 % margin of about ±6 points, so 89 % against 87 % is not a real difference.
- **Experiment 2.** The comparison uses a fixed row-major order, since an MRV order would make the three modes identical. At 25 givens, 10 of 100 plain-BT runs reached the 200,000-assignment cap, so its average is a lower bound. Times depend on the CPU, and AC-3's fixed cost makes BT + FC + AC-3 slower on easy puzzles.
- **Search.** Reported times include computing h. The CSP inside the goal test has its own caps (20,000 assignments, 2 seconds) and treats hitting one as "not a goal"; this never happened in the full run. Greedy is not optimal.
- **Choices that depart from the brief.** The 45,075 / 9,002 split (Section 2) and exact-k injection are our rules. The brief's literal injection rule is available behind a flag, but we report only the exact version.

9. Conclusions and future work

A network that is wrong about 1.5 % of its digits can still feed a reliable Sudoku solver, if the solver knows how unsure the network was. AC-3 finds the contradiction, a search tries the cheapest corrections first, and a CSP solver checks each candidate. Together they restore the exact solution in 87–95 % of puzzles with 1–3 misreads.

- **The probabilities are the key ingredient.** Without correction the exact solution is never recovered once a digit is misread. Random correction recovers at most 28 %, and even random choices restricted to the search's own candidates recover only 66 %, 9 % and 0 % at 1, 2 and 3 misreads, against 95 %, 89 % and 87 % for cost-ordered search.
- **A* is optimal and expands fewer nodes**, but on a correction space of at most 13,276 states UCS is faster in seconds. We did not test a larger space, where the node savings would matter more.
- **Each CSP technique pays for itself on hard puzzles.** Forward checking, AC-3 and MAC each cut the assignments needed at 25 givens, though AC-3's fixed cost is not repaid on easy puzzles.
- **The main weakness is silent wrong answers.** A corrected puzzle with exactly one solution can still be the wrong one, in 5 to 13 % of puzzles with 1–3 misreads and in 40 % of puzzles with 4.

Future work follows from these limits. First, the system could report how sure it is, for example by comparing the cost of the best correction set with the next-best set that also gives a uniquely solvable puzzle; a small gap would flag exactly the answers we found to be wrong. We have not built this. Second, the CNN could be retrained with several seeds and trained longer, to separate the variation caused by training from the variation caused by puzzles. Third, the system could be tested on photographed Sudoku, which needs grid detection and handles noise and skew. Fourth, varying the number of candidate cells and the correction budget would show how recovery trades against search effort, in particular for 4 misreads.

References

- Course brief: Search + CSP + Neural Network Student Projects, project P7 and the common checklist.

- Guo, C., Pleiss, G., Sun, Y. and Weinberger, K. Q. (2017). On calibration of modern neural networks. Proceedings of the 34th International Conference on Machine Learning.
- Hart, P. E., Nilsson, N. J. and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.
- LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324.
- Mackworth, A. K. (1977). Consistency in networks of relations. Artificial Intelligence, 8(1), 99–118.
- Russell, S. and Norvig, P. (2020). Artificial Intelligence: A Modern Approach, 4th edition. Pearson.